Matrix opeartions

In [1]:
```
import numpy as np
```

In [3]:
```
#create a 1-D array and dimension
v1=np.array([5,4,7,8])
print(v1)
print(v1.ndim)
```

```
[5 4 7 8]
1
```

In [6]:
```
#create a 2-D array
v2=np.array([[1,5,3],[4,7,9],[4,9,4]])
print(v2)
```

```
[[1 5 3]
 [4 7 9]
 [4 9 4]]
```

In [7]:
```
#create a matrix of 2 rows 3 column
v3=np.array([[1,5],[4,7],[4,9]])
print(v3)
```

```
[[1 5]
 [4 7]
 [4 9]]
```

In [13]:
```
#matrix addition
v4=np.array([[6,3,8],[2,5,2],[8,1,3]])
print(v4)
print('addition of v2 and v4 is \n',v2+v4)
```

```
[[6 3 8]
 [2 5 2]
 [8 1 3]]
addition of v2 and v4 is
 [[ 7  8 11]
 [ 6 12 11]
 [12 10  7]]
```

In [9]:
```
#matrix subtraction
print('subtraction of v2 and v4 is \n',v2-v4)
```

```
subtraction of v2 and v4 is
 [[-5  2 -5]
 [ 2  2  7]
 [-4  8  1]]
```

In [10]:
```
# scalar matrix
print('scalar matrix of v4 is \n',2*v4)
```

```
scalar matrix of v4 is
 [[12  6 16]
 [ 4 10  4]
 [16  2  6]]
```

In [12]:
```
# multiplication matrix
print(np.dot(v2,v4))
#element wise multiplication just use '*' for multiply.
```

```
[[ 40  31  27]
 [110  56  73]
 [ 74  61  62]]
```

In [14]:
```
#transpose of the matrix
print('transpose of matrix v2 is \n',v2.T)
```

```
transpose of matrix v2 is
 [[1 4 4]
 [5 7 9]
 [3 9 4]]
```

In [24]:
```
#matrix determinant
a=np.array([[1,2,3],[1,1,5],[2,4,7]])
print('Matrix a= \n',a)
print('determinant of the a is',np.linalg.det(a))
```

```
Matrix a=
 [[1 2 3]
 [1 1 5]
 [2 4 7]]
determinant of the a is -1.0
```

In [22]:
```
#inverse of the matrix
print('inverse of the a is \n',np.linalg.inv(a))
```

```
inverse of the a is
 [[13.  2. -7.]
 [-3. -1.  2.]
 [-2. -0.  1.]]
```

In [25]:
```
b= np.array([[1,2,3],
             [1,1,5],
             [2,4,6]])

if np.linalg.det(b) != 0:
    print('inverse of a is \n',np.linalg.inv(b))
else:
    print('inverse does not exist for matrix having deteminant zero ')
```

```
inverse does not exist for matrix having deteminant zero
```

In [26]:
```
# identity matrix
print('identity matrix \n',np.eye(3)) #parameter is used to define the dimension of matri
```

```
identity matrix
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Image Processing

In [27]:
```
import matplotlib.pyplot as plt
from skimage import color,io
from skimage.transform import resize
```

In [41]:
```
image=io.imread('C:/Users/USER-11/Downloads/images.jpg')
#print(image)
gray_image=color.rgb2gray(image)
gray_image=resize(gray_image,(256,256))
```

In [35]:
```
#brighting the image
bright_image=gray_image*2.0
bright_image=np.clip(bright_image,0,1)
```

In [66]:
```
#noisy or distortion will be present in the image
noisy_image=gray_image+0.3
noisy_image=np.clip(noisy_image,0,1)
```

In [37]:
```
#darkening the image
dark_image=gray_image-0.2
dark_image=np.clip(dark_image,0,1)
```

In [39]:
```
#fliping the image
transpose_image=gray_image.T
```

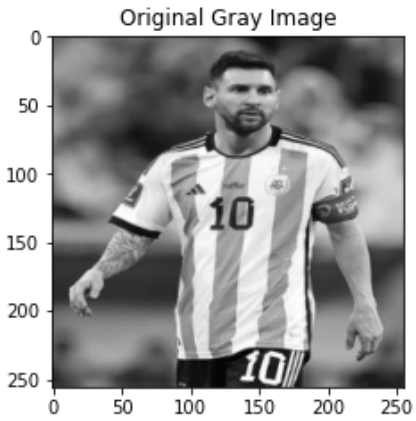
In [40]:
```
#inversing the image
inverse_image=np.linalg.inv(gray_image)
inverse_image=np.clip(inverse_image,0,1)
```

In [50]:
```
#plotting the original image
plt.imshow(image, cmap='gray')
plt.title('original image')
plt.show()
```



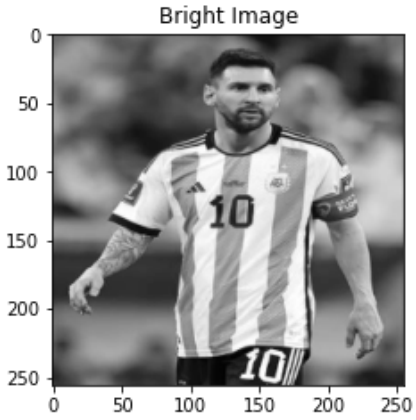
In [55]:
```
plt.figure(figsize=(12,12))
plt.subplot(2,3,1)
plt.imshow(gray_image,cmap='gray')
plt.title('Original Gray Image')
```

Out[55]: Text(0.5, 1.0, 'Original Gray Image')



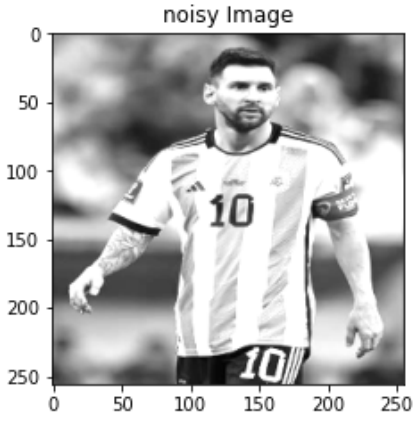
In [61]:
```
plt.figure(figsize=(12,12))
plt.subplot(2,3,2)
plt.imshow(bright_image,cmap='gray')
plt.title('Bright Image')
```

Out[61]: Text(0.5, 1.0, 'Bright Image')



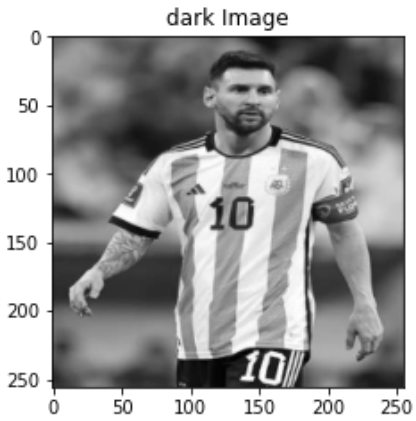
In [67]:
```
plt.figure(figsize=(12,12))
plt.subplot(2,3,3)
plt.imshow(noisy_image,cmap='gray')
plt.title('noisy Image')
```

Out[67]: Text(0.5, 1.0, 'noisy Image')



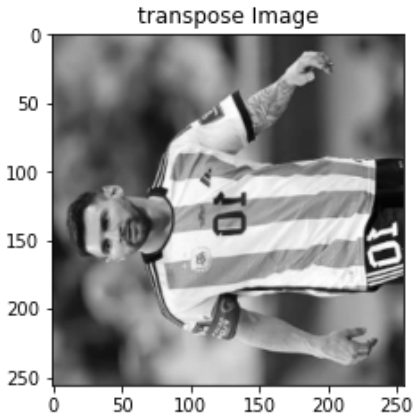
In [58]:
```
plt.figure(figsize=(12,12))
plt.subplot(2,3,4)
plt.imshow(dark_image,cmap='gray')
plt.title('dark Image')
```

Out[58]: Text(0.5, 1.0, 'dark Image')



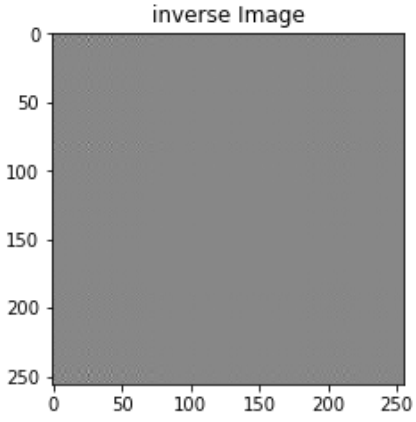
In [59]:
```
plt.figure(figsize=(12,12))
plt.subplot(2,3,5)
plt.imshow(transpose_image,cmap='gray')
plt.title('transpose Image')
```

Out[59]: Text(0.5, 1.0, 'transpose Image')



In [60]:
```
plt.figure(figsize=(12,12))
plt.subplot(2,3,6)
plt.imshow(inverse_image,cmap='gray')
plt.title('inverse Image')
```

Out[60]: Text(0.5, 1.0, 'inverse Image')



In [ ]: